



**HOCHSCHULE
MITTWEIDA**
University of Applied Sciences

BACHELOR THESIS

Mr.
Georgii Burdin

**Detecting Model Drift in
LLM-as-a-Service APIs: A Benchmarking
Framework for Campaign-Relevance
Classification**

Mittweida, April 2026



Faculty of **Applied Computer Sciences and Biosciences**

BACHELOR THESIS

Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification

Author:

Georgii Burdin

Course of Study:

B.Sc. Applied Mathematics

Seminar Group:

MA22w1-B

First Examiner:

Prof <Name> <First Examiner>

Second Examiner:

<Second Examiner>

Submission:

Mittweida, 01.04.2026

Defense/Evaluation:

Mittweida, 2026

Bibliographic Description

Burdin, Georgii:

Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification. – 2026. – 16 S.

Mittweida, Hochschule Mittweida – University of Applied Sciences, Faculty of Applied Computer Sciences and Biosciences, Bachelor Thesis, 2026.

Referat

1. Context

Gocomo uses API-based large language models (LLMs) for campaign-relevance classification on social-media posts.

2. Practical Problem

These API models change over time without full transparency, so the performance of a deployed classification pipeline may degrade.

3. Operational Constraint

Re-running a full benchmark frequently is too expensive.

4. Methodological Problem

Therefore, a framework is needed that can:

- represent the task consistently,
- monitor performance over time with a smaller but informative sample,
- detect drift statistically,
- support cost-aware model selection,
- and possibly recover performance through prompt optimisation.

Contents

Contents	I
List of Figures and Tables	III
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.3 Research Questions	2
1.4 Task Definition	2
1.4.1 What do we do?	2
1.4.2 Example LLM Request Payload	2
2 Data and Evaluation Setup	4
2.1 Formal Problem Definition	4
2.2 The Reference Benchmark vs. Operational Constraints	4
2.3 Informative Sampling via Model Disagreement	4
2.3.1 Geometry of the Decision Boundary	5
2.3.2 Proof of Maximal Uncertainty via Disagreement	5
2.4 Statistical Derivation of the Sample Size	6
2.5 The Role of Selection Bias and Importance Weighting	7
2.5.1 The Horvitz–Thompson Link	7
2.5.2 Operational Utility: Signal Amplification	8
3 Drift Monitoring and Model Selection Methodology	9
3.1 Formalisation of Temporal Evaluation	9
3.2 Statistical Drift Detection via McNemar’s Test	9
3.2.1 The Contingency Table	9
3.2.2 The Test Statistic and Alerting Rule	9
3.3 Cost-Aware Model Selection	10
3.3.1 The Deployment Utility Function	10
3.3.2 Decision Logic for Pipeline Adaptation	10
4 Prompt Optimisation as an Adaptive Mechanism	11
4.1 Integrating Optimisation into the Monitoring Loop	11
4.2 Pareto-Based Prompt Search (GEPA)	11
5 System Architecture and Experiments	12
5.1 Overview of the Evaluation Pipeline	12
5.2 Dataset Storage and Trace Logging	12
5.3 Batch Evaluation Process	12
5.4 Interaction with LLM APIs	13
5.5 Metrics Collection and Monitoring Dashboard	14
6 Conclusion	16
Bibliography	17

Statutory Declaration in Lieu of an Oath

18

List of Figures and Tables

List of Figures

5.1 High-level campaign-relevance evaluation and monitoring pipeline. 12

1 Introduction

1.1 Context and Motivation

Industrial classification pipelines increasingly rely on Large Language Model (LLM) providers that expose their systems through application programming interfaces. In such settings, the deployed model can be viewed as a non-stationary black-box function $f_t(x)$ whose internal parameters and update schedule are controlled by the provider rather than by the user. As a result, the behaviour of the same API-accessible model may change over time even when the downstream task, the prompt, and the evaluation procedure remain fixed [1–3].

The practical motivation for this thesis arises from the author’s two-year work experience at **Gocomo GmbH**. In the company’s social-media analytics workflow, LLMs are used in batch mode to determine whether a post is relevant to a specified campaign, brand, event, or other marketing entity. This is naturally formulated as a binary classification task. Because the volume of incoming content is large, manual verification is not scalable, and automated classification becomes a necessary operational component.

However, the use of external LLM APIs creates a reliability problem. A model that performs well during an initial benchmark may later degrade due to silent provider-side updates. In a business-critical pipeline, such degradation must be detected early. At the same time, repeated full-scale benchmarking on a large labelled dataset is costly. This creates a need for a monitoring methodology that is statistically informative, operationally feasible, and economically justified.

1.2 Problem Statement

This thesis addresses the problem of how to evaluate and monitor API-based LLM classifiers in a production environment where model behaviour may change over time. The central challenge is that the user does not control the underlying model updates, yet must still make reliable deployment decisions based on observed task performance.

More specifically, the problem consists of four interconnected parts:

1. The social-media relevance detection task must be formalised in a way that permits consistent evaluation across models, prompts, and time points.
2. Because repeated evaluation on the full reference benchmark is economically expensive, a smaller monitoring subset must be constructed without losing the ability to detect practically relevant performance changes.
3. A statistical rule is required to distinguish variability in performance from genuine model drift.
4. Once multiple candidate models are monitored, deployment decisions must account not only for predictive performance but also for inference cost.

In addition, if a deployed model exhibits degraded performance, it is natural to ask whether this degradation can be mitigated without immediately replacing the model. This motivates the investigation of prompt optimisation as an adaptive mechanism within the monitoring framework.

Accordingly, the thesis does not study model drift merely as an observational phenomenon. Its objective is to develop a practically usable framework for benchmark design, drift-sensitive monitoring, cost-aware model selection, and post-drift adaptation for binary classification with black-box LLM APIs.

1.3 Research Questions

The problem stated above is investigated through the following research questions:

- RQ1:** How can the campaign-relevance detection task be formally represented as a binary classification problem for non-stationary black-box LLM APIs?
- RQ2:** How can a reduced monitoring subset be derived from a larger reference benchmark so that routine evaluation remains substantially cheaper while preserving sensitivity to model drift?
- RQ3:** Which statistical criteria and alerting rules are appropriate for detecting practically significant performance degradation over time?
- RQ4:** How should candidate models be compared and selected when deployment decisions depend on both predictive quality and inference cost?
- RQ5:** To what extent can prompt optimisation improve or recover task performance after drift has been detected, and how can such optimisation be integrated into the monitoring loop?

1.4 Task Definition

1.4.1 What do we do?

At a high level, the production pipeline takes each incoming social-media post and asks an LLM whether the content is relevant to a given marketing campaign. The model sees a structured description of the campaign (brands, slogans, product lines) together with the post's text, hashtags, and selected video frames, and must decide if at least one of the campaign entities is mentioned, visually present (e.g. logo), or otherwise clearly implied. The LLM returns a binary decision for every post in the batch, which is then used downstream for filtering and reporting.

1.4.2 Example LLM Request Payload

During evaluation, each social-media post is converted into a structured request sent to the LLM provider API. The request consists of a system prompt describing the classification task together with user inputs containing the extracted content of the post (text, metadata, and video frames).

A simplified example payload is shown below.

NB!: MAKE A VISUALISATION IN FIGMA OF THE PAYLOAD WITH A IMAGE → STRUCTURED OUTPUT, it is literally a map function

```
[
  {
    "role": "system",
    "content": "Classify whether the post is relevant to the campaign  
'Douglas_Beauty Day 2025'. Return TRUE or FALSE and explain the reasoning."
  },
  {
    "role": "user",
    "content": "Transcription: ..."
  },
  {
    "role": "user",
    "content": "Hashtags: []"
  },
  {
    "role": "user",
    "content": [
      {
        "type": "input_text",
        "text": "Video frames extracted from the post"
      },
      {
        "type": "input_image",
        "image_url": "https://.../frame1.jpg"
      },
      {
        "type": "input_image",
        "image_url": "https://.../frame2.jpg"
      }
    ]
  }
]
```

The system prompt defines the classification criteria and the list of campaign brands. The user messages contain the multimodal content of the post, including captions, transcriptions, hashtags, and selected video frames. Results are restricted to binary response of is this post relevant or not to the requested topic

To evaluate model drift rigorously, we must first define the classification task within a probabilistic framework.

2 Data and Evaluation Setup

2.1 Formal Problem Definition

To evaluate model drift rigorously, we must first define the classification task within a probabilistic framework. Let $\mathcal{X} \subset \mathbb{R}^d$ denote the high-dimensional input space of social-media posts (comprising text, metadata, and visual features) and $\mathcal{Y} = \{0, 1\}$ the binary label space, where 1 indicates relevance to a campaign.

In the context of LLM-as-a-Service, the model is a non-stationary black box. We define the classifier as a function $f(x; \pi, \theta_t) \rightarrow \hat{y} \in \{0, 1\}$, where π is a fixed prompt and θ_t represents the hidden, time-varying model weights.

Under the zero-one loss function $\mathcal{L}(\hat{y}, y) = \mathbb{I}(\hat{y} \neq y)$, the **Expected Risk** (or true error rate) at time t is:

$$R_t(f) = \mathbb{E}_{(x,y) \sim P_t} [\mathbb{I}(f(x; \pi, \theta_t) \neq y)] \quad (2.1)$$

Since the true data-generating distribution P_t is unknown, we must estimate this risk empirically.

2.2 The Reference Benchmark vs. Operational Constraints

Let $\mathcal{D}_N$ be the reference benchmark consisting of $N = 1200$ manually labelled examples, establishing the ground truth. Evaluating all candidate models against $\mathcal{D}_N$ yields a highly precise empirical risk $\hat{R}_N$.

However, running inference on $N = 1200$ instances weekly across multiple candidate models is economically prohibitive. We require a monitoring subset $\mathcal{D}_n \subset \mathcal{D}_N$ of size $n \ll N$. The central mathematical challenge is to select $\mathcal{D}_n$ such that it remains highly sensitive to temporal shifts in the parameters θ_t , despite the significantly reduced sample size.

2.3 Informative Sampling via Model Disagreement

A simple random sample (SRS) of size n would yield an unbiased estimator of R_t . However, the vast majority of social-media posts are "easy" instances located far from the model's decision boundary, where the conditional probability $P(Y = 1|X) \approx 1$ or 0 . These instances exhibit low prediction variance and are highly robust to small parameter shifts in θ_t .

To maximize the sensitivity of our monitoring subset, we abandon uniform sampling in favor of **Uncertainty Sampling**. We construct $\mathcal{D}_n$ exclusively from instances where an initial ensemble of distinct models $\mathcal{M} = \{f_1, f_2, \dots, f_K\}$ exhibited disagreement:

$$\mathcal{D}_n = \{x_i \in \mathcal{D}_N \mid \exists f_j, f_k \in \mathcal{M} : \hat{y}_{i,j} \neq \hat{y}_{i,k}\} \quad (2.2)$$

To understand why this maximizes detection power, we must formalize the geometry of the classifier's decision space and prove that disagreement isolates instances of maximal statistical variance.

2.3.1 Geometry of the Decision Boundary

In modern representation learning, it is standard to assume the *Manifold Hypothesis*: the tenet that nominally high-dimensional data are in fact concentrated near a lower-dimensional submanifold [4, 5]. This assumption serves as a primary inductive bias for deep learning architectures [6], as it allows models to circumvent the curse of dimensionality by focusing on the intrinsic geometric structure of the data rather than the ambient space.

Assumption 1 (Data Manifold)

Let the input space $\mathcal{X} \subset \mathbb{R}^d$ be a smooth, compact Riemannian manifold.

For any ensemble $\mathcal{M}$, let $\bar{p} : \mathcal{X} \rightarrow [0, 1]$ be the continuous consensus probability function, where $\bar{p}(x)$ represents the ensemble's expected probability that $Y = 1$.

Definition 2.1 (Consensus Decision Boundary)

The decision boundary $\mathcal{B}$ of the ensemble is the pre-image of the classification threshold:

$$\mathcal{B} = \bar{p}^{-1}(0.5) = \{x \in \mathcal{X} \mid \bar{p}(x) = 0.5\} \quad (2.3)$$

Proposition 1

*If 0.5 is a regular value of the function $\bar{p}$ (i.e., the gradient $\nabla \bar{p}(x) \neq 0$ for all $x \in \mathcal{B}$), then by the **Regular Level Set Theorem**, the decision boundary $\mathcal{B}$ is a smooth submanifold of $\mathcal{X}$ of codimension 1.*

This establishes $\mathcal{B}$ as a formal geometric hypersurface dividing the data manifold $\mathcal{X}$ into two distinct classification regions.

2.3.2 Proof of Maximal Uncertainty via Disagreement

We now prove that sampling instances where models disagree mathematically confines the sample to a tight neighborhood around the submanifold $\mathcal{B}$.

Assumption 2 (Bounded Ensemble Divergence)

Assume the models in $\mathcal{M}$ represent competent classifiers approximating the same underlying logic, such that their individual scoring functions $p_m(x)$ diverge from the consensus $\bar{p}(x)$ by no more than ϵ :

$$\sup_{x \in \mathcal{X}} |p_m(x) - \bar{p}(x)| \leq \epsilon \quad \text{for all } m \in \{1, \dots, K\} \quad (2.4)$$

Proposition 2 (Disagreement Bounds Distance to $\mathcal{B}$)

If there exists a disagreement within the ensemble $\mathcal{M}$ for a given instance x (i.e., $p_j(x) < 0.5$ and $p_k(x) \geq 0.5$), then the consensus probability $\bar{p}(x)$ is strictly bounded within an ϵ -neighborhood of 0.5.

Proof: Applying the bounded divergence assumption to the disagreeing models:

$$\bar{p}(x) - \epsilon \leq p_j(x) < 0.5 \implies \bar{p}(x) < 0.5 + \epsilon \quad (2.5)$$

$$\bar{p}(x) + \epsilon \geq p_k(x) \geq 0.5 \implies \bar{p}(x) \geq 0.5 - \epsilon \quad (2.6)$$

Thus, it strictly follows that $\bar{p}(x) \in [0.5 - \epsilon, 0.5 + \epsilon]$. $\square$

By geometrically isolating instances where models produce conflicting predictions, we are effectively sampling directly from the ϵ -tube surrounding the submanifold $\mathcal{B}$. At the theoretical limit ($\epsilon \rightarrow 0$), the binary entropy $H(Y|X)$ reaches its global maximum of 1 bit. The model is maximally uncertain, meaning the expected accuracy approaches random guessing ($A \approx 0.5$).

This geometric isolation formalizes why disagreement sampling forces the evaluation subset into the **Maximum Variance** state for a Bernoulli process, which serves as the foundational assumption for determining the required sample size.

2.4 Statistical Derivation of the Sample Size

The decision to sample from the ϵ -neighborhood of the decision boundary directly dictates the required sample size n . By forcing the expected accuracy toward $A = 0.5$, we deliberately subject our evaluation to the **Maximum Entropy** state. For a Bernoulli process, the variance $\sigma^2 = A(1 - A)$ reaches its absolute maximum of 0.25 exactly at $A = 0.5$.

NB! show disagreement of annotators

To determine n under this worst-case variance, we must define the **Minimum Detectable Effect (MDE)** Δ . In our production environment, human inter-annotator disagreement establishes an irreducible noise floor of $\epsilon_{noise} \approx 0.05$ (5%). Any model drift smaller than ϵ_{noise} is statistically indistinguishable from background label noise. Thus, our monitoring system must have a margin of error strictly bounded by $\Delta = 0.05$.

We formalize this requirement using the margin of error for a one-sided hypothesis test (detecting degradation) at a significance level of $\alpha = 0.05$. Let $Z_{1-\alpha} \approx 1.645$ be the critical value from the standard normal distribution. The required sample size n must satisfy:

$$Z_{1-\alpha} \sqrt{\frac{A(1-A)}{n}} \leq \Delta \quad (2.7)$$

Substituting the maximum variance assumption $A(1 - A) = 0.25$ and our noise floor $\Delta = 0.05$, we obtain:

$$1.645 \sqrt{\frac{0.25}{n}} \leq 0.05 \quad (2.8)$$

$$\frac{0.8225}{\sqrt{n}} \leq 0.05 \implies \sqrt{n} \geq 16.45 \quad (2.9)$$

$$n \geq 270.6 \quad (2.10)$$

Voluntarily, the theoretically required sample size of $n \geq 271$ is rounded up to $n = 300$. This calculation proves that 300 is the strict mathematical minimum required to detect a 5% model drift under the maximum variance conditions induced by our informative sampling strategy.

2.5 The Role of Selection Bias and Importance Weighting

Because $\mathcal{D}_{300}$ intentionally over-represents the manifold's decision boundary, the empirical accuracy calculated directly on this subset, $\hat{A}_n$, is a **biased estimator** of the global population accuracy A_N . Formally, $\mathbb{E}[\hat{A}_n] < A_N$, as the subset is deliberately composed of instances with high epistemic uncertainty.

In standard statistical evaluation, selection bias is viewed as a flaw to be corrected. In the context of our drift monitoring framework, however, this bias is an operational requirement. We track the biased metric $\hat{A}_n$ because it isolates the exact region of the input space where temporal parameter shifts ($\Delta\theta_t$) manifest most strongly, thereby maximizing the statistical power of the alerting rule.

Nevertheless, it is necessary to mathematically guarantee that this heavily biased subset remains formally linked to the underlying data-generating distribution, rather than being an arbitrary or disconnected sample. This rigorous link is established through the theory of **Importance Sampling** and the **Change of Measure**.

2.5.1 The Horvitz-Thompson Link

Let $S_i \in \{0, 1\}$ be an indicator variable for the inclusion of the i -th reference instance in the monitoring subset, with probability $q_i = P(S_i = 1 \mid \text{disagreement})$. The global accuracy A_N can be theoretically recovered from the biased sample using the **Horvitz-Thompson (HT) estimator**:

$$\hat{A}_{HT} = \frac{1}{N} \sum_{i=1}^N \frac{S_i \cdot \mathbb{I}(\hat{y}_i = y_i)}{q_i} = \frac{1}{N} \sum_{i \in \mathcal{D}_n} \frac{\mathbb{I}(\hat{y}_i = y_i)}{q_i} \quad (2.11)$$

Here, the inverse probability $1/q_i$ acts as an importance weight (or Radon-Nikodym derivative) that maps the biased sampling measure back to the uniform population measure.

Proof of Unbiasedness: To prove that $\hat{A}_{HT}$ correctly recovers the global accuracy, we take its expectation over the sampling design:

$$\mathbb{E}[\hat{A}_{HT}] = \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N \frac{S_i \cdot \mathbb{I}(\hat{y}_i = y_i)}{q_i} \right] \quad (2.12)$$

By the linearity of expectation, and noting that the true labels and model predictions at a fixed time t are deterministic with respect to the sampling mechanism:

$$\mathbb{E}[\hat{A}_{HT}] = \frac{1}{N} \sum_{i=1}^N \frac{\mathbb{I}(\hat{y}_i = y_i)}{q_i} \mathbb{E}[S_i] \quad (2.13)$$

Since S_i is a Bernoulli random variable, its expected value is exactly its probability of occurrence, $\mathbb{E}[S_i] = q_i$. Substituting this yields:

$$\mathbb{E}[\hat{A}_{HT}] = \frac{1}{N} \sum_{i=1}^N \frac{\mathbb{I}(\hat{y}_i = y_i)}{q_i} \cdot q_i = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i) = A_N \quad (2.14)$$

□

2.5.2 Operational Utility: Signal Amplification

The proof above confirms that $\mathcal{D}_{300}$ is a statistically valid basis for inference, as it remains anchored to the reference population through the HT estimator. However, within the operational monitoring loop, we deliberately report the raw empirical accuracy $\hat{A}_n$ rather than the unbiased estimate $\hat{A}_{HT}$.

The rationale for this choice is rooted in **Signal-to-Noise Ratio (SNR) optimization**. Applying the importance weights $1/q_i$ would mathematically suppress the frequency of errors occurring at the decision boundary, effectively "smoothing" the data to reconstruct the global mean. While this is desirable for population estimation, it is counter-productive for **anomaly detection**. By utilizing the raw biased metric, we maintain the magnification of errors where the model is most unstable. Thus, $\hat{A}_n$ functions as a high-gain monitoring signal that provides a leading indicator of model drift, while the HT-framework provides the necessary theoretical guarantee that this signal is a rigorous and mathematically reversible projection of the global performance.

3 Drift Monitoring and Model Selection Methodology

3.1 Formalisation of Temporal Evaluation

Let $t \in \{1, \dots, T\}$ represent discrete evaluation intervals (e.g., weekly batch runs). At each time step t , a deployed model m evaluates the fixed monitoring subset $\mathcal{D}_{300}$. We denote the vector of temporal predictions for the i -th instance as $\hat{y}_{i,t} \in \{0, 1\}$.

Because the evaluation is conducted on the exact same set of instances $\mathcal{D}_{300}$ across time, the longitudinal accuracy measurements $\hat{A}_{t-1}$ and $\hat{A}_t$ are strongly dependent. Consequently, standard statistical tests that assume independent samples (such as the two-proportion Z -test) are invalid, as they overestimate variance and reduce the statistical power to detect subtle model drift.

3.2 Statistical Drift Detection via McNemar's Test

To formally detect model drift, we shift the focus from the marginal accuracies to the **discordant pairs**—the specific instances where the model's behavior has changed between $t - 1$ and t .

3.2.1 The Contingency Table

We construct a 2×2 contingency table comparing the correctness of the model's predictions over time. Let $E_{i,t} = \mathbb{I}(\hat{y}_{i,t} = y_i)$ be the indicator of a correct prediction at time t . The frequencies of the four possible outcomes across the $n = 300$ instances are defined as:

- a : Concordant correct ($E_{i,t-1} = 1$ and $E_{i,t} = 1$)
- b : Discordant regression ($E_{i,t-1} = 1$ and $E_{i,t} = 0$)
- c : Discordant improvement ($E_{i,t-1} = 0$ and $E_{i,t} = 1$)
- d : Concordant incorrect ($E_{i,t-1} = 0$ and $E_{i,t} = 0$)

In this framework, the instances contributing to a and d provide no information regarding temporal shifts. The drift signal is isolated entirely within the off-diagonal elements b and c .

3.2.2 The Test Statistic and Alerting Rule

We establish the null hypothesis $H_0 : p_b = p_c$, which states that the marginal probabilities of success are equal across time, implying no model drift. The alternative hypothesis for degradation is $H_a : p_b > p_c$.

To test this, we utilize **McNemar's test statistic** with a continuity correction, which follows a χ^2 distribution with 1 degree of freedom:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \quad (3.1)$$

The automated monitoring pipeline triggers a **Statistical Drift Alert** at time t if two conditions are met:

1. **Directionality:** $b > c$ (the model "broke" more instances than it "fixed").
2. **Significance:** The p -value derived from the χ^2 statistic is less than the specified confidence level $\alpha = 0.05$.

3.3 Cost-Aware Model Selection

When a drift alert is triggered, or when evaluating new models for deployment, predictive accuracy cannot be considered in isolation from inference cost. Let $C_t(m)$ represent the API inference cost (in USD per 1,000 requests) and $A_t(m)$ the empirical accuracy for model $m \in \mathcal{M}$.

3.3.1 The Deployment Utility Function

We formulate model selection as a multi-objective optimization problem by defining a **Deployment Utility Function** $J_t(m)$ that penalizes excessive operational expenditure:

$$J_t(m) = A_t(m) - \lambda \cdot \log(1 + C_t(m)) \quad (3.2)$$

where $\lambda \geq 0$ is a business-defined sensitivity parameter.

- When $\lambda = 0$, the system selects purely on predictive capability ($J_t = A_t$).
- When $\lambda > 0$, the logarithmic penalty bounds the impact of highly expensive models, reflecting the diminishing business returns of marginal accuracy improvements at exorbitant API costs.

3.3.2 Decision Logic for Pipeline Adaptation

The monitoring architecture follows an automated decision tree executed upon every batch completion:

1. **Monitor:** Calculate the McNemar χ^2 statistic for the currently deployed model m_{curr} against its baseline performance at t_0 .
2. **Detect:** If $p < 0.05$ and $b > c$, register a significant degradation event.
3. **Act:** Evaluate the utility $J_t(m)$ for all available fallback models. The pipeline dynamically routes future traffic to the model $m^* = \arg \max_m J_t(m)$. If no fallback model satisfies the baseline utility threshold, the framework initiates the **Prompt Optimisation** loop (detailed in Chapter 4) to attempt performance recovery without altering the underlying model API.

4 Prompt Optimisation as an Adaptive Mechanism

4.1 Integrating Optimisation into the Monitoring Loop

When a deployed model triggers a drift alert ($D_t < -\tau_{\text{drift}}$), switching to a more expensive model is not the only recourse. We can attempt to recover performance via prompt optimization.

4.2 Pareto-Based Prompt Search (GEPA)

We utilize a reflective evolutionary method (GEPA) [7] for prompt adaptation. Because prompt space is discrete and non-convex, greedy selection often fails. Instead, we maintain a candidate pool of prompts and evaluate their per-instance score vectors $\mathbf{s}(\Pi) = (s_1(\Pi), \dots, s_n(\Pi))$.

We retain prompts based on Pareto dominance to preserve search diversity. A prompt $\Pi^{(a)}$ dominates $\Pi^{(b)}$ if it performs at least as well on all instances and strictly better on at least one. By mutating prompts on the Pareto frontier using natural-language reflections from execution traces, we can systematically search for a prompt Π^* that maximizes $J_t(m)$ without altering the underlying API model.

5 System Architecture and Experiments

5.1 Overview of the Evaluation Pipeline

The experiments are implemented as an automated batch evaluation pipeline that connects a versioned Langfuse dataset, multiple LLM providers (OpenAI, Gemini, Claude), and a monitoring/analysis layer. Figure 5.1 shows the high-level flow from labeled examples to batch jobs, responses, and metrics.

High-level flow from labeled examples in Langfuse through provider-specific batch jobs to structured responses and monitoring metrics.

Figure 5.1: High-level campaign-relevance evaluation and monitoring pipeline.

Throughout this chapter, the *primary* metrics of interest are:

- **Classification accuracy** on the monitoring subset
- **Inference cost** per model (measured via token usage and provider pricing)

The core idea is that every evaluation run is:

1. Defined as a subset of the campaign relevance dataset,
2. Executed as provider-native batch jobs, and
3. Ingested back into Langfuse as traces with structured outputs and scores, so that drift can be analysed across time and models.

5.2 Dataset Storage and Trace Logging

All labels and input features for the campaign relevance task are stored in Langfuse as a dedicated dataset. Each row represents a single influencer post with a stable `custom_id`, campaign brief, caption, optional image references, and a boolean `campaign_relevant` label. During evaluation, each model call is recorded as a Langfuse trace whose metadata includes the `custom_id`, model name, provider, run identifier, latency, and token usage. This makes it possible to reconstruct historical evaluations and to filter traces by dataset version, model, or time window when analysing drift.

5.3 Batch Evaluation Process

Batch runs are orchestrated by `run_eval.py`, which:

1. Loads an evaluation subset via `load_dataset_rows`, either directly from Langfuse or from a CSV export (e.g. the disagreement subset).
2. Applies a shared prompt template to produce a normalised conversational payload (system + user messages) per example.
3. Calls provider-specific helpers (`batch_openai.py`, `batch_gemini.py`, `batch_claude.py`) that write JSONL request files and submit batch jobs.

4. Writes a manifest data/batch_ids.json listing all submitted batch IDs, models, and providers.

GitHub Actions workflows schedule and execute this pipeline: they trigger batch submission, poll job status until completion, and then run the ingestion scripts (`poll_and_ingest.py`, `ingest.py`) that attach responses to the correct traces in Langfuse.

5.4 Interaction with LLM APIs

All models are queried via provider-native batch APIs, but the logical payload is always the same: a schema-constrained campaign relevance call built from one labeled example. Concretely:

- **OpenAI** (`batch_openai.py`): writes JSONL lines of the form `{custom_id, method="POST", url="/v1/responses", body}` and submits the file to the OpenAI Batch API with `endpoint="/v1/re`
- **Gemini** (`batch_gemini.py`): writes JSONL lines with a request object (contents, optional `systemInstruction`, `generationConfig`) and uploads the file to the Google GenAI batch API via `client.batches.create(model=..., src=...)`.
- **Claude** (`batch_claude.py`): builds an in-memory list of `{custom_id, params}` records and passes it directly to the Anthropic Messages Batches API via `client.messages.batches.create`; the streamed batch results are later normalised into JSONL by `download_claude_batch_results`.

In all three cases, the underlying request encodes:

- a `custom_id` linking back to the dataset row,
- a conversational payload (system instructions + user content with campaign brief and post),
- and a JSON schema (`CampaignRelevanceMinimal`) that forces the response into a `{campaign_relevant, relevancy_reasoning}` object.

Listing 5.1 and Listing 5.2 give a concrete example of the batch line and output schema used in the experiments.

Listing 5.1: Example OpenAI batch JSONL line for the campaign relevance task.

```
{
  "custom_id": "run_2025-02-10_example_001",
  "method": "POST",
  "url": "/v1/responses",
  "body": {
    "model": "gpt-5.1-mini",
    "input": [
      {
        "role": "system",
        "content": "You are a classifier that decides whether a social-media post"
      },
      {
        "role": "user",
        "content": "Campaign: Douglas_Beauty Day 2025\nCaption: ... \nHashtags: [...]"
      }
    ],
    "response_format": {
      "type": "json_schema",
      "json_schema": {
```

```

    "name": "CampaignRelevanceMinimal",
    "schema": {"$ref": "#/definitions/CampaignRelevanceMinimal"},
    "strict": true,
    "definitions": {
      "CampaignRelevanceMinimal": {
        "type": "object",
        "properties": {
          "campaign_relevant": {"type": "boolean"},
          "relevancy_reasoning": {"type": "string"}
        },
        "required": ["campaign_relevant", "relevancy_reasoning"],
        "additionalProperties": false
      }
    }
  }
}

```

Listing 5.2: Minimal JSON schema used to constrain model outputs in the campaign relevance experiments.

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "CampaignRelevanceMinimal",
  "type": "object",
  "properties": {
    "campaign_relevant": {
      "type": "boolean",
      "description": "TRUE if the post is relevant to the campaign, otherwise FALSE"
    },
    "relevancy_reasoning": {
      "type": "string",
      "description": "Short natural-language explanation of the decision."
    }
  },
  "required": ["campaign_relevant", "relevancy_reasoning"],
  "additionalProperties": false
}

```

5.5 Metrics Collection and Monitoring Dashboard

The monitoring stack exposes exactly two core metrics: **classification accuracy** on the monitoring subset and **inference cost** per model.

Ingestion is handled by the script `ingest.py`, which processes each batch output file as follows:

1. Parse each JSONL line and normalise the provider response (OpenAI, Gemini, Claude) into a common structure.
2. Extract the boolean `campaign_relevant` prediction from the model output.

3. Look up the ground-truth label for the corresponding `custom_id` from Langfuse or a CSV export.
4. Derive token usage (input, output, total) from the provider-specific usage fields.
5. Convert token usage into a dollar cost using the per-million-token prices in `config/models.yaml`.
6. Write a Langfuse trace/span with:
 - metadata: `model`, `provider`, `custom_id`, optional `run_id`;
 - the parsed input and raw provider response;
 - three scores:
 - `accuracy` $\in \{0, 1\}$ (per-trace correctness),
 - `error_type` $\in \{\text{true_positive}, \text{false_negative}, \text{true_negative}, \text{false_positive}\}$,
 - `cost_usd` (total dollar cost for this trace);
 - detailed `usage_details` and `cost_details` for debugging and audits.

On top of this logging layer, a small Streamlit dashboard queries Langfuse and aggregates scores over the 300-example monitoring subset. The dashboard provides:

- per-model accuracy curves over time (mean of the per-trace accuracy scores),
- confusion-matrix summaries derived from `error_type`,
- request volume and latency distributions per run,
- and cost views such as cost per run and estimated cost per 1,000 inferences.

Screenshots from this Streamlit dashboard are used in the experiments chapter to show concrete drift events and the associated accuracy–cost trade-offs. Together, `ingest.py` and the dashboard form the practical monitoring surface of the statistical surveillance loop described in the previous chapters.

6 Conclusion

This thesis provides a statistically grounded, cost-aware framework for the continuous evaluation of LLMs in production. By decoupling the initial reference benchmark from an informative monitoring subset, and pairing statistical drift detection with automated prompt optimization, organizations can maintain high classification performance despite the inherent instability of black-box LLM APIs.

Bibliography

- [1] L. Chen, M. Zaharia, and J. Zou, "How is ChatGPT's behavior changing over time?", *arXiv preprint arXiv:2307.09009*, 2023.
- [2] S. Tu et al., "ChatLog: Carefully evaluating the evolution of ChatGPT across time", *arXiv preprint arXiv:2304.14106*, 2023.
- [3] J. Echterhoff et al., "MUSCLE: A model update strategy for compatible LLM evolution", *arXiv preprint arXiv:2407.09435*, 2024.
- [4] L. Cayton, "Algorithms for manifold learning", University of California, San Diego, Tech. Rep. CS2008-0923, 2005.
- [5] C. Fefferman, S. Mitter, and H. Narayanan, "Testing the manifold hypothesis", *Journal of the American Mathematical Society*, vol. 29, no. 4, pp. 983–1049, 2016.
- [6] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013.
- [7] Author1, Author2, Author3, et al., "Gepa: Reflective prompt evolution can outperform reinforcement learning", *arXiv preprint arXiv:2507.19457*, 2025.

Statutory Declaration in Lieu of an Oath

I – Georgii Burdin – do hereby declare in lieu of an oath that I have composed the presented work independently on my own and without any other resources than the ones given.

All thoughts taken directly or indirectly from external sources are correctly acknowledged.

This work has neither been previously submitted to another authority nor has it been published yet.

Mittweida, 12. March 2026

Location, Date

Georgii Burdin